



南開大學
Nankai University

计算机学院
大数据计算及应用实验报告

实验 1: PageRank 计算大作业

姓名：王旭

学号：2312166

专业：计算机科学与技术

2026 年 5 月 13 日

目录

1 摘要	3
2 实验目的与要求依据	3
3 数据集说明	3
3.1 文件来源、路径与提交约定	3
3.2 记录格式与编码	3
3.3 定量统计的获取方式	4
3.4 图模型与基本规模	4
3.5 出度与入度分布	4
3.6 Dead-end	5
3.7 数据特点与程序实现的对应	5
4 算法原理	5
4.1 问题背景	5
4.2 邻接矩阵与列随机转移矩阵	5
4.3 带 Teleport 的随机冲浪迭代	6
4.4 Dead-end（出度为零）的修正	6
4.5 Spider-trap 与 Teleport 的作用	6
4.6 收敛判据与幂迭代	6
5 程序实现与关键代码	7
5.1 文件组织与执行流程	7
5.2 全局常量（与作业参数对齐）	7
5.3 边表读入 <code>read_edges</code>	7
5.4 顶点表与 CSR 构图 <code>build_index_and_csr</code>	8
5.4.1 紧凑下标: <code>index_of_sorted</code>	8
5.4.2 顶点集 <code>node_ids</code> （排序去重）	8
5.4.3 出度 <code>outdeg</code> 与 CSR 行指针 <code>row_ptr</code>	8
5.4.4 填充列索引 <code>col_idx</code>	9
5.5 PageRank 迭代 <code>pagerank_iterate</code>	9
5.5.1 初值与每轮 teleport、dead-end 底项	9
5.5.2 分块稀疏乘加（沿边推送）	10
5.5.3 L1 收敛判断与向量交换	10
5.6 Top-K 输出 <code>write_top_k</code>	11
5.7 主程序 <code>main</code> 与资源回收	11
5.8 课程要求、原理与实现对照	12
6 内存与性能优化结果	13
6.1 已采用措施	13
6.2 自测方法（ <code>memoryuse-cpp.py</code> ）与结果记录	13
6.3 并行（OpenMP）	16

7 编译、运行与提交说明	17
7.1 编译环境与命令	17
7.2 运行方式与输出	17
8 实验结果与分析	18
8.1 Top 100 结果与分析	18
8.2 收敛性	19
8.3 不同 teleport 参数对比	19
9 总结与心得	21
10 参考文献	21

课程实验报告：PageRank 计算大作业

1 摘要

本实验在给定有向图数据集 `Data.txt` 上实现 **PageRank** 计算。图以边表形式存储（每行 `FromNodeID ToNodeID`），程序采用 **稀疏 CSR（压缩稀疏行）** 表示转移结构，并在矩阵-向量乘阶段按 **源节点分块** 遍历，以满足课程对 **稀疏矩阵** 与 **分块矩阵** 的硬性要求。迭代采用带随机跳转（teleport）的模型，**跳转概率 α 取 0.85**（与作业要求一致），对 **dead-end（出度为 0 的节点）** 按均匀跳转处理；**spider-trap** 由 teleport 项缓解。迭代在 **L1 范数** 小于阈值时停止，输出 **Top 100 节点** 至 `Res.txt`（格式：`NodeID Score`）。实现为 **C++ 单文件**，使用 **g++ 静态链接** 生成 `PageRank.exe`；按课程下发的 `memoryuse-cpp.py` 测得 **10 次运行 RSS 峰值的算术平均约 2.85 MB**、**墙钟时间算术平均约 0.12 s**，满足作业给出的资源导向约束（约 80 MB、60 s）。没有使用 `networkx.pagerank` 等现成 PageRank API。

关键词：PageRank 稀疏矩阵（CSR） 分块 dead-end teleport 静态链接

2 实验目的与要求依据

根据作业所给的实验要求文档 `requirement.pdf`，本实验需完成：

- 读取 `Data.txt`，计算 PageRank，输出 **Top 100 及分数** 至 `Res.txt`（`NodeID Score`）。
- **必须给出 teleport（跳转）参数为 0.85 时的结果**；可对其它参数做对比。
- 在基础 PageRank 之外，须进行 **内存优化**，且 **稀疏矩阵与分块矩阵** 为 **强制要求**。
- 考虑 **dead-end** 与 **spider-trap**；迭代 **直至收敛**；**禁止使用现成 PageRank API**（如 Python 的 `networkx.pagerank`）。
- 资源上建议：峰值内存 **低于约 80 MB**，运行时间 **在 60 s 内**；并行/分布式为可选项且非主线。

3 数据集说明

3.1 文件来源、路径与提交约定

- 文件名称：`Data.txt`。
- 文件位置：`pagerank 大作业\Data.txt`（与 `requirement.pdf` 同目录）。

3.2 记录格式与编码

- **每行语义**：一条有向边。行首整数为 **起点（FromNodeID）**，行末整数为 **终点（ToNodeID）**，与作业说明「`FromNodeID ToNodeID`」一致。
- **分隔符**：两列之间为空白字符（空格或制表符均可；本实现使用 `operator>>` 按流读取整数，与常见 ASCII 边表兼容）。

- **无表头**：文件从第一行即为数据行，无列名行、无注释行。
- **编码与体积**：文件为纯数字与换行构成的文本；本机测得文件大小约 **1.54 MB**（约 **1616971** 字节），共 **150 000** 行，与边数一致，便于估计读入时间与内存上界（边表阶段为 $O(E)$ 条 `int` 记录）。

3.3 定量统计的获取方式

各表中的数值均通过对 `Data.txt` 的顺序扫描与统计得到，**未改写**原始数据文件。也可用 `Python`、`Excel` 等按相同逻辑复现。

统计内容	获取思路
行数 / 边数 $ E $	换行计数或逐行读入计数
顶点数 $ V $	将每行两端点置入集合后统计元素个数
编号范围与未出现编号	在出现过的 ID 上取最小、最大值，并与连续编号区间对比
重复有序对	用集合记录 (u, v) ，判断该行是否为首次出现的边
自环	统计满足 $u = v$ 的边数
出度 / 入度及 min、max、均值	对每条边执行 <code>out[u]++</code> 、 <code>in[v]++</code> ，再对所有顶点聚合
dead-end 个数	在顶点集合上统计出度为 0 的顶点个数

3.4 图模型与基本规模

将 `Data.txt` 建模为简单有向图 $G = (V, E)$ ：顶点集 V 为至少在边中出现一次的 `NodeID` 集合；边集 E 为所有有序对 (u, v) 。

统计项	符号/含义	数值
边数（行数）	$ E $	150 000
顶点数	$ V $	9 500
NodeID 取值范围	最小 ~ 最大编号	0~9 999
编号是否占满区间	—	否；0~9 999 共 10 000 个槽位中仅 9 500 个在图中出现，有 500 个编号未作为顶点出现
重复边	—	0 （150 000 行对应 150 000 个互不相同的 (u, v) 有序对）
自环 (u, u)	—	15 条（占边总数约 0.01% ；实现上仍按普通出边参与归一化与 CSR 存储）

平均度数：有向图下，总出度之和等于总入度之和，均等于边数 150000。故平均出度、平均入度均为

$$\frac{150000}{9500} \approx 15.79.$$

3.5 出度与入度分布

对顶点 $v \in V$ ，记出度 $d_{\text{out}}(v)$ 为以 v 为起点的边数，入度 $d_{\text{in}}(v)$ 为以 v 为终点的边数。在本数据集上统计得到：

指标	出度 d_{out}	入度 d_{in}
最小值	0 (dead-end)	2
最大值	36	32
均值（约）	15.79	15.79

解读要点：

1. **入度最小为 2**：每个出现在图中的顶点至少被两条边指向，不存在入度为 0 的顶点。
2. **出度最大 36、入度最大 32**：存在度数较高的顶点；**Top 100** 中排名靠前的节点常与较大入度或较密集的局部结构有关。
3. **出度可为 0**：对应 **dead-end**；本数据中有 1 000 个，约占顶点总数的 10.5%，迭代中须单独处理。

3.6 Dead-end

定义：若 $v \in V$ 且 $d_{\text{out}}(v) = 0$ ，则称 v 为 **dead-end**（死链/死端）。在经典随机冲浪模型中，若不在矩阵层面做修正，游走在到达 v 后将无法定义「沿随机出边」的下一步，因此必须在 PageRank 迭代中把这部分概率质量以 **均匀跳转**等方式分配到全图顶点。

本数据：dead-end 顶点个数为 1 000，占顶点总数 9 500 的比例约为

$$\frac{1000}{9500} \approx 10.53\%.$$

本图含较多 **dead-end**，迭代实现中须显式处理。

3.7 数据特点与程序实现的对应

- **顶点重映射**：NodeID 为 $0 \dots 9999$ 中的稀疏子集，程序中不直接以 10000 维稠密向量存秩，而是将出现的 ID 排序去重后映射为 $0 \dots n-1$ ($n = 9500$)，降低存储并与 CSR 行号一致。
- **稀疏存储**： $|E| = 1.5 \times 10^5 \ll n^2$ ，必须使用边表或 CSR 等**稀疏结构**，避免 $O(n^2)$ 稠密矩阵，对应 **Sparse Matrix** 要求。
- **分块计算**：迭代中对 CSR 做乘加时按源顶点分块遍历，对应 **Block Matrix** 要求。

4 算法原理

4.1 问题背景

PageRank 将 Web 抽象为有向图，超链接为有向边，用「随机冲浪者」的长期行为刻画页面重要性：若随机游走按给定规则在图上运行足够长时间，各顶点被访问的**平稳比例**即 PageRank。本实验在给定的 `Data.txt` 上执行同一思想，输出平稳向量中分值最高的 **Top 100** 顶点。

4.2 邻接矩阵与列随机转移矩阵

设邻接矩阵 $\mathbf{A} \in \mathbb{R}^{n \times n}$ ，其中 $A_{ji} = 1$ 当且仅当存在有向边 (i, j) ，否则为 0。记顶点 i 的出度为

$$d_i = \sum_{j=0}^{n-1} A_{ji}.$$

若 $d_i > 0$, 从 i 沿出边均匀随机走一步, 则转移到 j 的概率为 A_{ji}/d_i 。将所有列堆叠, 得到列随机矩阵 $\mathbf{M}$ (每列元素和为 1):

$$M_{ji} = \frac{A_{ji}}{d_i}, \quad d_i > 0.$$

当 $d_i = 0$ 时, 经典 PageRank 将第 i 列替换为全 $1/n$, 表示到达死端后下一步在整个顶点集上均匀分布; 本程序用等价的 `dead_mass` 分摊实现。

4.3 带 Teleport 的随机冲浪迭代

引入跳转概率 $\alpha \in (0, 1)$: 每一步以概率 α 按矩阵 $\mathbf{M}$ 沿出边转移 (若有出边), 以概率 $1 - \alpha$ 不跟随链接, 在整个 n 个顶点上均匀随机选取下一顶点。记 $\mathbf{1}$ 为全 1 向量, $\mathbf{e} = \mathbf{1}/n$ 。则一步更新为

$$\mathbf{r}^{(t+1)} = (1 - \alpha)\mathbf{e} + \alpha\mathbf{M}^\top \mathbf{r}^{(t)} = \frac{1 - \alpha}{n}\mathbf{1} + \alpha\mathbf{M}^\top \mathbf{r}^{(t)}.$$

其中 $\mathbf{r}^{(t)} \in \mathbb{R}^n$ 为第 t 轮各顶点的 PageRank 估计 (列向量), $(\mathbf{M}^\top \mathbf{r})_j = \sum_i M_{ji} r_i$ 表示从父顶点 i 经出边流入 j 的贡献。本实现将 α 设定为 0.85, 故 teleport 强度为 $0.15/n$, 与 `requirement.pdf` 一致。

4.4 Dead-end (出度为零) 的修正

若 $d_i = 0$, 则上式中 $\mathbf{M}$ 的第 i 列在原定义下不存在。标准做法是将该列设为 $1/n$ 的常数向量: 一旦质量落在 i , 下一步以均匀分布散布到所有顶点。代数上等价于: 设 dead-end 集合 $D = \{i : d_i = 0\}$, 记

$$\text{dead_mass} = \sum_{i \in D} r_i^{(t)}, \quad \text{dead_share} = \frac{\alpha \cdot \text{dead_mass}}{n},$$

则对每个顶点 j , 在写入本轮新向量时先赋予公共底

$$r_j^{(t+1)} \leftarrow \frac{1 - \alpha}{n} + \text{dead_share},$$

再对 $d_i > 0$ 的顶点 i , 向每个出边邻居 j 累加 $\alpha r_i^{(t)}/d_i$ 。这与「dead-end 列全为 $1/n$ 」后再做 $\alpha\mathbf{M}^\top \mathbf{r}$ 在数学上一致, 且便于与 CSR 按行存出边的数据结构结合。

4.5 Spider-trap 与 Teleport 的作用

Spider-trap 指若干顶点形成强连通子结构, 随机游走进入后难以仅靠局部边转移泄漏到图外, 导致质量在陷阱内过度堆积。Teleport 项 $(1 - \alpha)/n$ 使每一步都有至少 $1 - \alpha$ 的概率跳到全图均匀分布, 从而削弱陷阱的封闭性, 使迭代趋向稳定的主分布。本实验未调用任何现成 PageRank API, 完全按上述显式迭代计算。

4.6 收敛判据与幂迭代

`requirement.pdf` 要求迭代至收敛。本实现采用幂迭代: 初值取 $\mathbf{r}^{(0)} = \mathbf{e}$ (各分量为 $1/n$), 反复应用 4.3 的更新式。停机准则为相邻两轮向量的 L1 距离

$$\|\mathbf{r}^{(t+1)} - \mathbf{r}^{(t)}\|_1 = \sum_{j=0}^{n-1} |r_j^{(t+1)} - r_j^{(t)}| < \varepsilon,$$

其中 ε 取 10^{-10} 量级。采用 **L1** 范数便于实现与观察。另设 **最大迭代次数** 上界，防止数值异常时长时间运行。

在 $\alpha \in (0, 1)$ 且修正后的整体转移为随机矩阵的前提下，幂迭代在标准条件下收敛到 **Perron 向量**（主特征向量）；本实验以 **L1 残差** 为实际操作中的「收敛」定义。

5 程序实现与关键代码

5.1 文件组织与执行流程

- **单文件提交**: 我们的实现文件为 `PageRank.cpp`, 仅依赖 **C++ 标准库** (`<algorithm>`、`<vector>`、`<fstream>` 等), 未使用 Eigen、`networkx.pagerank` 等第三方 PageRank 或稀疏代数库, 满足要求中「自行实现」与「少用第三方库」的要求。
- **匿名命名空间**: 第 26 ~ 250 行将 `read_edges`、`build_index_and_csr`、`pagerank_iterate`、`write_top_k` 等封装在匿名命名空间内, 限制符号作用域, 避免与链接单元中其他目标文件冲突。
- **main 入口** (第 252 ~ 308 行): 解析命令行与环境变量 $\rightarrow$ 读边 $\rightarrow$ 构图 $\rightarrow$ 迭代 $\rightarrow$ 写 `Res.txt`。

5.2 全局常量（与作业参数对齐）

以下常量集中定义于第 58 ~ 64 行:

```
1 // PageRank.cpp 第 58 - 64 行
2 constexpr double kAlpha = 0.85;
3 constexpr double kL1Tol = 1e-10;
4 constexpr int kMaxIter = 1000000;
5 constexpr int kTopK = 100;
6 constexpr int kSourceBlock = 256;
```

- **kAlpha**: 即第 4 节中的 α , 固定为 **0.85**, 对应 `requirement.pdf` 必须提交的 teleport 结果。
- **kL1Tol** / **kMaxIter**: **L1 收敛阈值**与迭代安全上界, 对应「迭代至收敛」。
- **kTopK**: 输出 **Top 100**。
- **kSourceBlock**: **CSR 乘向量时外层按源点分块的块大小**, 对应 **Block Matrix** 要求。

5.3 边表读入 `read_edges`

第 80 ~ 102 行从给定路径打开文本文件, 按 `int` 顺序读入 `u v`。失败时输出错误路径; 在我们的 Windows 系统下通过 `write_stderr_utf8_line` 打印 UTF-8 中文用法 (第 85 ~ 88 行), 避免控制台代码页导致的乱码。成功时对 `edges` 做 `reserve(160000)`, 减少扩容次数。

```
1 // PageRank.cpp 第 95 - 101 行
2 edges.reserve(160000);
3 while (in >> u >> v) {
4     edges.push_back({u, v});
5 }
6 return !edges.empty();
```


5.4 顶点表与 CSR 构图 `build_index_and_csr`

构图函数位于第 107~148 行, 与第 4.2 节列随机矩阵 M 的非零结构一致; 实现上采用 CSR (压缩稀疏行), 满足 Sparse Matrix 的要求。

5.4.1 紧凑下标: `index_of_sorted`

```
1 // PageRank.cpp 第 66-73 行
2 inline int index_of_sorted(const std::vector<int>& ids, int id) {
3     const auto it = std::lower_bound(ids.begin(), ids.end(), id);
4     if (it == ids.end() || *it != id) {
5         return -1;
6     }
7     return static_cast<int>(it - ids.begin());
8 }
```

- **作用:** 在已排序的唯一 `NodeID` 表 `ids` 中, 将题目给出的原始编号映射为 $0 \dots n-1$ 的紧凑下标, 供 `outdeg`、`row_ptr`、`col_idx` 及秩向量 `r` 下标使用。
- **复杂度:** 单次 $O(\log n)$; 构图阶段每条边对端点各查一次, 总 $O(|E| \log n)$, 在 $|E|$ 与 n 同量级时可接受。
- **与数据的关系:** 在第 3 节中我们知道 `NodeID` 在 $0 \sim 9999$ 中稀疏出现; 若用「最大 ID 长度」开数组会浪费且不利于更大编号, 故采用有序表与二分法。

5.4.2 顶点集 `node_ids` (排序去重)

```
1 // PageRank.cpp 第 112-120 行
2 std::vector<int> endpoints;
3 endpoints.reserve(edges.size() * 2);
4 // ... 遍历 edges, 将每条边的 u、v 依次 push_back 到 endpoints
5 std::sort(endpoints.begin(), endpoints.end());
6 endpoints.erase(std::unique(endpoints.begin(), endpoints.end()), endpoints.end());
7 node_ids.swap(endpoints);
```

- `reserve(edges.size() * 2)`: 为所有端点预留空间, 减少 `push_back` 扩容, 属实现层面的内存与性能细节。
- `sort` + `unique` + `erase`: 得到升序、无重复的顶点表, 长度 $n = |V|$, 即第 3 节统计的顶点数。
- **输出:** `Res.txt` 中需写出题目给定的 **原始** `NodeID`, 因此保留 `node_ids[i]` 与紧凑下标 i 的对应关系, 输出时用 `node_ids[idx]` 还原。

5.4.3 出度 `outdeg` 与 CSR 行指针 `row_ptr`

```
1 // PageRank.cpp 第 122-137 行
2 const int n = static_cast<int>(node_ids.size());
3 outdeg.assign(static_cast<size_t>(n), 0);
4 for (const auto& e : edges) {
```

```

5     const int i = index_of_sorted(node_ids, e.u);
6     // ... j = index_of_sorted(node_ids, e.v) 仅用于合法性; 出度只记在源点 i
7     outdeg[static_cast<size_t>(i)] += 1;
8 }
9 // ... std::vector<int> cur(n, 0); 由 outdeg 做前缀和写入 row_ptr[0..n]
10 // ... E = row_ptr[n]; col_idx.assign(E, 0);

```

- `outdeg[i]`：即第 4.2 节中的出度 d_i ，用于归一化系数 $1/d_i$ 及判断 **dead-end** ($d_i = 0$)。
- `row_ptr`：经典 CSR 前缀和，`row_ptr[i]` 到 `row_ptr[i+1]-1` 为顶点 i 的所有出边在 `col_idx` 中的下标区间；记边数 $E = |E|$ ，则 `row_ptr[n]` 等于 E 。
- **稀疏性**：仅存长度为 $n+1$ 的 `row_ptr` 与长度为 E 的 `col_idx`，空间 $O(n+E)$ ，**不显式存储** $n \times n$ 稠密邻接矩阵，对应 **Sparse Matrix**。

5.4.4 填充列索引 `col_idx`

```

1 // PageRank.cpp 第 141-147 行
2 for (const auto& e : edges) {
3     const int si = index_of_sorted(node_ids, e.u);
4     const int sj = index_of_sorted(node_ids, e.v);
5     const int pos = row_ptr[static_cast<size_t>(si)] + cur[static_cast<size_t>(si)];
6     col_idx[static_cast<size_t>(pos)] = sj;
7     cur[static_cast<size_t>(si)] += 1;
8 }

```

- `cur[si]`：在填第 si 行时，记录该行当前已写入的条数，与 `row_ptr[si]` 相加得到写入位置，保证同一源点的出边在 `col_idx` 中**连续**，便于按行（按源点）分块遍历。
- `sj` 为紧凑下标：`col_idx` 中存的是列索引（目标顶点在稠密秩向量中的下标），与 `pagerank_iterate` 中 `r_new[j]` 的下标一致。
- 与矩阵 M 的对应：非零元 $M_{ji} = 1/d_i$ 的值在迭代时按 `push` 现算，结构上由 (i, j) 在 CSR 中的出现位置隐含表示。

5.5 PageRank 迭代 `pagerank_iterate`

5.5.1 初值与每轮 teleport、dead-end 底项

```

1 // PageRank.cpp 第 159-178 行
2 const int n = static_cast<int>(outdeg.size());
3 const double inv_n = 1.0 / static_cast<double>(n);
4 const double teleport = (1.0 - kAlpha) * inv_n;
5
6 std::fill(r.begin(), r.end(), inv_n);
7
8 for (int iter = 0; iter < kMaxIter; ++iter) {
9     double dead_mass = 0.0;
10    // ... 扫描 i: 若 outdeg[i]==0 则 dead_mass += r[i]
11    const double dead_share = kAlpha * dead_mass * inv_n;
12
13    for (int j = 0; j < n; ++j) {

```

```

14     r_new[static_cast<size_t>(j)] = teleport + dead_share;
15 }
16 // ... 紧接着为 (2) 中分块 CSR 与 (3) 中 L1、swap; 完整 for(iter) 见源文件

```

- `std::fill(r, inv_n)`: 初值 $r^{(0)} = 1/n$, 与第 4.3 节一致。
- `teleport`: 即 $(1 - \alpha)/n$, 由常量 `kAlpha = 0.85` 与 `inv_n` 计算得到。
- `dead_mass` / `dead_share`: 实现 dead-end 列全 $1/n$ 后与 α 相乘再均匀加到各分量的效果; 随后对每个 j 写入本轮 `r_new` 的公共底, 再叠加沿边项。
- 处理逻辑: 显式处理 dead-end, 并含 `teleport`, 对应 `requirement.pdf` 中 Consider of dead-ends and spider-traps 及必须报告 $\alpha=0.85$ 。

5.5.2 分块稀疏乘加 (沿边推送)

```

1 // PageRank.cpp 第 180-196 行
2 for (int bi = 0; bi < n; bi += kSourceBlock) {
3     const int i_end = std::min(n, bi + kSourceBlock);
4     for (int i = bi; i < i_end; ++i) {
5         const int od = outdeg[static_cast<size_t>(i)];
6         if (od == 0) {
7             continue;
8         }
9         const double push = kAlpha * r[static_cast<size_t>(i)] / static_cast<double>(od);
10        const int p0 = row_ptr[static_cast<size_t>(i)];
11        const int p1 = row_ptr[static_cast<size_t>(i + 1)];
12        // ... for (int p = p0; p < p1; ++p) { j = col_idx[p]; r_new[j] += push; }
13    }
14 }

```

- `kSourceBlock` 外层分块: 外层按源点 i 分段, 改善访问 `row_ptr`、`col_idx`、`r` 的局部性, 对应 Block Matrix 在计算组织上的要求 (在稀疏结构上按块做 SpMV)。
- `push = kAlpha * r[i] / od`: 即 $\alpha r_i^{(t)} / d_i$, 与第 4.2~4.3 节矩阵 M 的非零元与向量乘法一致。
- `od==0` 时 `continue`: 该源点质量已经按 dead-end 规则摊入 `r_new`, 此处不再沿边推送, 避免重复计数。

5.5.3 L1 收敛判断与向量交换

```

1 // PageRank.cpp 第 198-213 行
2 double l1 = 0.0;
3 // ... 累加 |r_new[i] - r[i]| 得到 L1 残差
4 r.swap(r_new);
5 if (l1 < kL1Tol) {
6     // ... 可选 verbose 打印后 break
7 }
8 // ... 若 iter+1 == kMaxIter 则输出警告
9 }

```

- **L1 残差**: $\|r^{(t+1)} - r^{(t)}\|_1$, 与第 4.6 节及 `kL1Tol` 一致; 小于阈值则 `break`, 满足 `iterate until reaching convergence`。
- `r.swap(r_new)`: 本轮结果成为下一轮输入, 双缓冲避免额外拷贝。
- `kMaxIter` 分支: 未收敛时的保护性输出, 便于调试; 正常数据下不应触发。

5.6 Top-K 输出 `write_top_k`

```

1 // PageRank.cpp 第 221 - 247 行
2 k = std::min(k, n);
3 std::vector<int> ord(static_cast<size_t>(n));
4 std::iota(ord.begin(), ord.end(), 0);
5 const auto cmp = [&](int a, int b) {
6     const double ra = rank[static_cast<size_t>(a)];
7     const double rb = rank[static_cast<size_t>(b)];
8     if (ra != rb) {
9         return ra > rb;
10    }
11    return node_ids[static_cast<size_t>(a)] < node_ids[static_cast<size_t>(b)];
12 };
13 std::partial_sort(ord.begin(), ord.begin() + k, ord.end(), cmp);
14 ord.resize(static_cast<size_t>(k));
15
16 std::ofstream out(path);
17 // ...
18 out << std::fixed << std::setprecision(8);
19 for (int i = 0; i < k; ++i) {
20     const int idx = ord[static_cast<size_t>(i)];
21     out << node_ids[static_cast<size_t>(idx)] << ' '
22         << rank[static_cast<size_t>(idx)] << '\n';
23 }

```

- `std::min(k, n)`: 顶点数不足 k 时不会越界, 健壮性处理。
- 比较器 `cmp`: 先按 `rank` 降序, 同分按 `node_ids` 升序, 保证 Top 100 次序稳定、可复现, 便于与同伴或参考结果对照。
- `partial_sort`: 时间约 $O(n \log k)$, 在 n 很大时优于整体排序, 属实现层面的效率考虑。
- 输出格式: NodeID、空格、分数, 小数 8 位, 与 `requirement.pdf` 中 `Res.txt` 的 `NodeID Score` 要求一致; 此处 k 取全局常量 `kTopK` (100)。

5.7 主程序 `main` 与资源回收

```

1 // PageRank.cpp 第 253 - 307 行 (节选)
2 int main(int argc, char** argv) {
3     #ifdef _WIN32
4         init_console_utf8();
5     #endif
6     std::ios::sync_with_stdio(false);
7     std::cin.tie(nullptr);
8
9     // ... 解析 -v/--verbose、默认或命令行指定的 in_file、out_file

```

```

10
11     std::vector<Edge> edges;
12     if (!read_edges(in_file, edges)) {
13         return 1;
14     }
15
16     std::vector<int> node_ids;
17     std::vector<int> row_ptr;
18     std::vector<int> col_idx;
19     std::vector<int> outdeg;
20     build_index_and_csr(edges, node_ids, row_ptr, col_idx, outdeg);
21     edges.clear();
22     edges.shrink_to_fit();
23
24     const int n = static_cast<int>(node_ids.size());
25     std::vector<double> r(static_cast<size_t>(n));
26     std::vector<double> r_new(static_cast<size_t>(n));
27
28     pagerank_iterate(row_ptr, col_idx, outdeg, r, r_new, log_verbose);
29
30     write_top_k(out_file, node_ids, r, kTopK);
31     // ... 可选 verbose 提示写出路径
32     return 0;
33 }

```

- 命令行与默认路径: 支持 `-v` / `--verbose` 与可选输入、输出路径, 默认 `Data.txt`、`Res.txt`, 便于本地与评测环境指定文件位置。
- `read_edges` → `build_index_and_csr`: 完成从边表到 CSR 的转换。
- `edges.clear(); edges.shrink_to_fit()`: 构图完成后释放边表多余容量, 降低峰值内存。
- `pagerank_iterate` → `write_top_k`: 完成 PageRank 与 Top 100 写出。

5.8 课程要求、原理与实现对照

本节用表格归纳 `requirement.pdf` 与第 4 节公式在代码中的对应关系, 并分为基本正确性、公式与符号线索两类。

(1) 基本要求 ↔ 理论 (第 4 节) ↔ 实现

requirement.pdf / 课程要点	第 4 节对应	实现落点 (第 5 节)
读取 <code>Data.txt</code> (边表)	4.2 图与 M 的结构	5.3 <code>read_edges</code> ; 默认/命令行路径 5.7
Sparse matrix	4.2 非零结构, 不显式稠密 M	5.4 CSR: <code>row_ptr</code> 、 <code>col_idx</code> 、 <code>outdeg</code>
Block matrix	4.3 乘加按块组织计算	5.2 <code>kSourceBlock</code> ; 5.5 (2) 外层 <code>bi</code> 分块
Dead-end 与 spider-trap	4.4、4.5	5.5 (1) <code>dead_mass</code> / <code>dead_share</code> ; 5.5 (1) <code>teleport</code> (α 跳转) 缓解陷阱
Teleport $\alpha=0.85$	4.3 式中 $(1-\alpha)/n$	5.2 <code>kAlpha</code> ; 5.5 (1) <code>teleport</code>
迭代至收敛	4.6 L1 判据	5.2 <code>kL1To1</code> 、 <code>kMaxIter</code> ; 5.5 (3) <code>l1</code> 与 <code>break</code>
<code>Res.txt</code> Top 100, NodeID Score	— (输出约定)	5.2 <code>kTopK</code> ; 5.6 <code>write_top_k</code> 、格式与稳定排序
禁止现成 PageRank API	4.5	5.1 仅 C++ 标准库; 无 Eigen / <code>networkx.pagerank</code> 等
静态链接可执行文件 (提交物)	—	第 7.1 <code>-static</code> ; 与 5.7 <code>main</code> 为同一源码产物

(2) 公式符号 ↔ 代码线索

第 4 节符号 / 步骤	代码中主要线索
$\alpha, (1 - \alpha)/n$	kAlpha, teleport
dead-end 均匀分摊 $\alpha \cdot \text{dead_mass}/n$	dead_mass, dead_share 与对 r_new 的初值循环
$\alpha \mathbf{M}^T \mathbf{r}$ 的按行推送	push 与对 col_idx 的内层循环
分块外层循环	kSourceBlock, bi
L1 收敛、相邻两轮向量之差的 L1 范数与双缓冲交换	l1, kL1Tol, r.swap
紧凑下标 $0 \dots n - 1$	index_of_sorted, node_ids
Top 100 还原原始 NodeID	write_top_k 中 node_ids[idx]

6 内存与性能优化结果

本章说明内存与耗时方面的做法及自测结果。

6.1 已采用措施

优化类型	做法摘要	代码 / 章节
稀疏存储	$O(n + E)$ 空间, 无 $n \times n$ 稠密阵	5.4 CSR: row_ptr, col_idx, outdeg
分块访问	源点按块遍历, 改善缓存局部性	5.2 kSourceBlock; 5.5 (2)
顶点编号	有序表 + 二分映射, 避免按最大 ID 开桶	5.4 (1) (2)
读入与构图	edges.reserve; 端点表 reserve, 减少扩容	5.3, 5.4 (2)
峰值内存	构图后 edges.clear() 与 shrink_to_fit()	5.7
迭代缓冲	仅 r 与 r_new 双缓冲, 无额外全图矩阵	5.5, 5.7
Top-K 排序	partial_sort, 约 $O(n \log k)$	5.6
I/O 与诊断	sync_with_stdio(false); 可选 -v / 环境变量	5.7
并行策略	作业未强制并行; 本实现 未用 OpenMP	6.3

6.2 自测方法 (memoryuse-cpp.py) 与结果记录

(1) 测量原理

课程下发的 memoryuse-cpp.py 借助 psutil 在子进程存活期间轮询 RSS, 得到单次运行中的峰值驻留内存 (MB) 与整次运行的墙钟时间 (s)。脚本对每次重新编译得到的可执行文件默认连续执行 10 次; 每轮结束控制台输出该轮的 Maximum 与 Time elapsed。10 次结束后, 在同目录 memory_usage_log.txt 末尾追加 avg mm used: ...MB, 即 10 次峰值 RSS 的算术平均。

(2) 运行依赖与编译参数

脚本依赖 Python 3 与 psutil。脚本从环境变量 CXXFLAGS 读取传给 g++ 的编译选项, 其内置默认值为 -fopenmp -mavx2; 本实现未使用 OpenMP, 且提交物须全静态链接 (requirement.pdf: Compile with all Static link library), 因此我们的自测如果要与提交物一致, CXXFLAGS 应与 g++ 与源文件名之间的选项串一致 (含 -static 与 UTF-8 相关选项)。脚本在拼接命令末尾仍会附加 -O2, 与上述串中已有的 -O2 重复不影响链接结果。

(3) 测试命令

在我们的 pagerank 大作业 文件夹下的终端中执行。自第 3 个参数起会原样传给生成的 PageRank.exe。

```

1 # powershell下的命令
2 $env:CXXFLAGS = "-O2 -std=c++17 -static -finput-charset=UTF-8 -fexec-charset=UTF-8"
3 python memoryuse-cpp.py "Cpp作业工作区/源码/PageRank.cpp" "Data.txt" "Cpp作业工作区/实验结果/Res.txt"

```

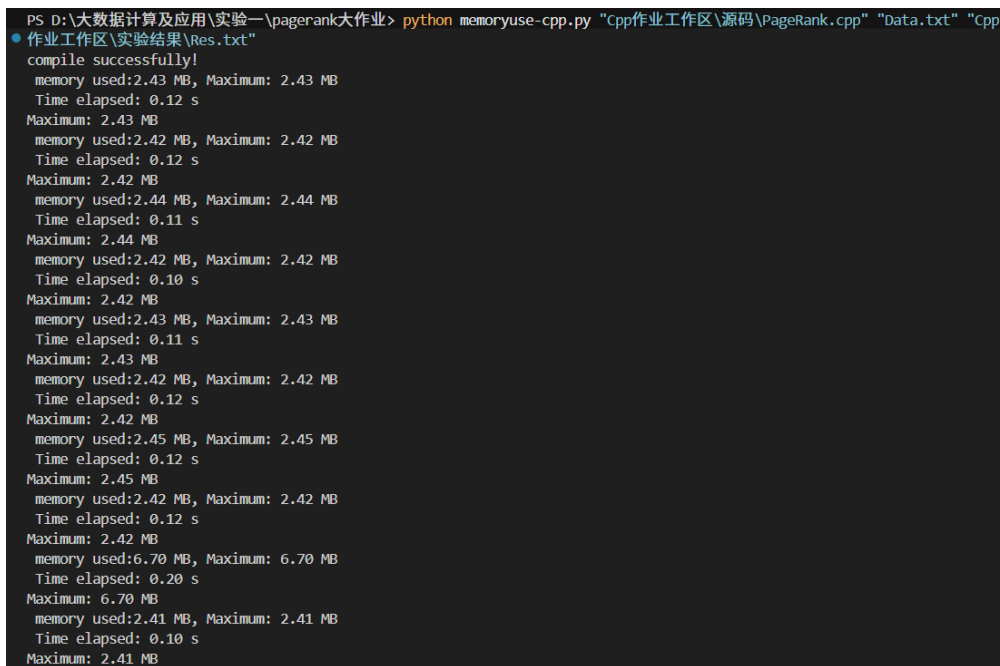
```

1 # cmd下的命令
2 set CXXFLAGS=-O2 -std=c++17 -static -finput-charset=UTF-8 -fexec-charset=UTF-8
3 python memoryuse-cpp.py "Cpp作业工作区/源码/PageRank.cpp" "Data.txt" "Cpp作业工作区/实验结果/Res.txt"

```

(4) 本组实测终端结果

以下为我们在一次自测中运行脚本得到的控制台输出：



```

PS D:\大数据计算及应用\实验一\pagerank大作业> python memoryuse-cpp.py "Cpp作业工作区\源码\PageRank.cpp" "Data.txt" "Cpp
● 作业工作区\实验结果\Res.txt"
compile successfully!
memory used:2.43 MB, Maximum: 2.43 MB
Time elapsed: 0.12 s
Maximum: 2.43 MB
memory used:2.42 MB, Maximum: 2.42 MB
Time elapsed: 0.12 s
Maximum: 2.42 MB
memory used:2.44 MB, Maximum: 2.44 MB
Time elapsed: 0.11 s
Maximum: 2.44 MB
memory used:2.42 MB, Maximum: 2.42 MB
Time elapsed: 0.10 s
Maximum: 2.42 MB
memory used:2.43 MB, Maximum: 2.43 MB
Time elapsed: 0.11 s
Maximum: 2.43 MB
memory used:2.42 MB, Maximum: 2.42 MB
Time elapsed: 0.12 s
Maximum: 2.42 MB
memory used:2.45 MB, Maximum: 2.45 MB
Time elapsed: 0.12 s
Maximum: 2.45 MB
memory used:2.42 MB, Maximum: 2.42 MB
Time elapsed: 0.12 s
Maximum: 2.42 MB
memory used:6.70 MB, Maximum: 6.70 MB
Time elapsed: 0.20 s
Maximum: 6.70 MB
memory used:2.41 MB, Maximum: 2.41 MB
Time elapsed: 0.10 s
Maximum: 2.41 MB

```

图 6.1: 终端输出结果

```

1 PS D:/大数据计算及应用/实验一/pagerank大作业> $env:CXXFLAGS = "-O2 -std=c++17 -static -finput-charset=UTF-8
-fexec-charset=UTF-8"
2 PS D:/大数据计算及应用/实验一/pagerank大作业> python memoryuse-cpp.py "Cpp作业工作区/源码/PageRank.cpp" "Data.txt"
"Cpp作业工作区/实验结果/Res.txt"
3 compile successfully!
4 memory used:2.43 MB, Maximum: 2.43 MB
5 Time elapsed: 0.12 s
6 Maximum: 2.43 MB
7 memory used:2.42 MB, Maximum: 2.42 MB
8 Time elapsed: 0.12 s
9 Maximum: 2.42 MB
10 memory used:2.44 MB, Maximum: 2.44 MB
11 Time elapsed: 0.11 s
12 Maximum: 2.44 MB
13 memory used:2.42 MB, Maximum: 2.42 MB
14 Time elapsed: 0.10 s
15 Maximum: 2.42 MB
16 memory used:2.43 MB, Maximum: 2.43 MB
17 Time elapsed: 0.11 s
18 Maximum: 2.43 MB

```

```

19 memory used:2.42 MB, Maximum: 2.42 MB
20 Time elapsed: 0.12 s
21 Maximum: 2.42 MB
22 memory used:2.45 MB, Maximum: 2.45 MB
23 Time elapsed: 0.12 s
24 Maximum: 2.45 MB
25 memory used:2.42 MB, Maximum: 2.42 MB
26 Time elapsed: 0.12 s
27 Maximum: 2.42 MB
28 memory used:6.70 MB, Maximum: 6.70 MB
29 Time elapsed: 0.20 s
30 Maximum: 6.70 MB
31 memory used:2.41 MB, Maximum: 2.41 MB
32 Time elapsed: 0.10 s
33 Maximum: 2.41 MB

```

(5) `memory_usage_log.txt` 对应片段
与上述 10 轮运行对应的部分如下。

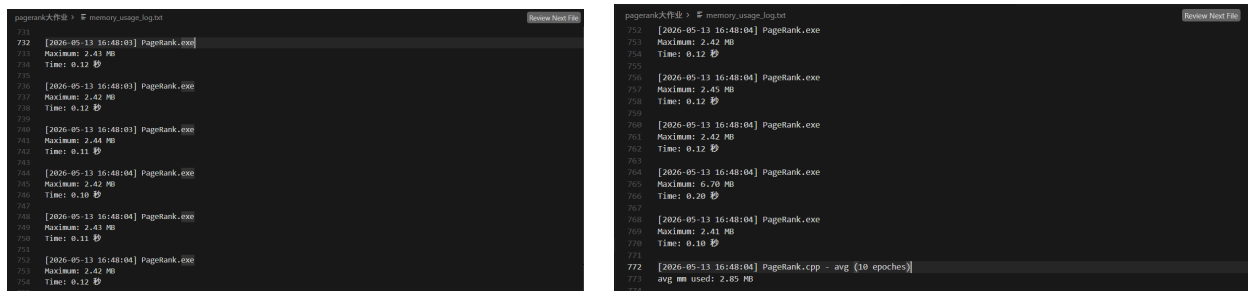


图 6.2: 终端输出结果

```

1 [2026-05-13 16:48:03] PageRank.exe
2 Maximum: 2.43 MB
3 Time: 0.12 秒
4
5 [2026-05-13 16:48:03] PageRank.exe
6 Maximum: 2.42 MB
7 Time: 0.12 秒
8
9 [2026-05-13 16:48:03] PageRank.exe
10 Maximum: 2.44 MB
11 Time: 0.11 秒
12
13 [2026-05-13 16:48:04] PageRank.exe
14 Maximum: 2.42 MB
15 Time: 0.10 秒
16
17 [2026-05-13 16:48:04] PageRank.exe
18 Maximum: 2.43 MB
19 Time: 0.11 秒
20
21 [2026-05-13 16:48:04] PageRank.exe
22 Maximum: 2.42 MB
23 Time: 0.12 秒
24
25 [2026-05-13 16:48:04] PageRank.exe

```



```

26 Maximum: 2.45 MB
27 Time: 0.12 秒
28
29 [2026-05-13 16:48:04] PageRank.exe
30 Maximum: 2.42 MB
31 Time: 0.12 秒
32
33 [2026-05-13 16:48:04] PageRank.exe
34 Maximum: 6.70 MB
35 Time: 0.20 秒
36
37 [2026-05-13 16:48:04] PageRank.exe
38 Maximum: 2.41 MB
39 Time: 0.10 秒
40
41 [2026-05-13 16:48:04] PageRank.cpp - avg (10 epoches)
42 avg mm used: 2.85 MB

```

(6) 我的本机的编译器版本

```

PS D:\大数据计算及应用\实验一\pagerank大作业> g++ --version
g++.exe (tdm64-1) 10.3.0
Copyright (C) 2020 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

PS D:\大数据计算及应用\实验一\pagerank大作业>

```

图 6.3: 本机编译器版本

```

1 PS D:/大数据计算及应用/实验一/pagerank大作业> g++ --version
2 g++.exe (tdm64-1) 10.3.0
3 Copyright (C) 2020 Free Software Foundation, Inc.
4 This is free software; see the source for copying conditions. There is NO
5 warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

```

(7) 汇总与说明

项目	数值
平均峰值内存 (MB)	2.85 MB (<code>memory_usage_log.txt</code> 中 <code>avg mm used</code>)
单次运行时间 (s)	0.12 s (10 次 <code>Time elapsed</code> 算术平均 0.122 s)
<code>g++</code> 版本	<code>g++.exe (tdm64-1) 10.3.0</code>

10 次中单次 RSS 峰值多数约 2.41~2.45 MB, 另有一次 6.70 MB、对应时间 0.20 s, 与 `psutil` 采样间隔及操作系统调度等因素下的瞬时读数抬升相符; 算术平均仍仅 2.85 MB, 远低于 80 MB 的导向阈值, 运行时间也远低于 60 s。

6.3 并行 (OpenMP)

我们的本实现 未使用 OpenMP, 以避免部分 Windows 环境下 `libgomp` 配置问题; 在现有数据规模下, 时间与内存已满足要求。

7 编译、运行与提交说明

7.1 编译环境与命令

(1) 正式编译命令

```
g++ -O2 -std=c++17 -static -finput-charset=UTF-8 -fexec-charset=UTF-8 PageRank.cpp -o PageRank.exe
```

(2) 主要编译选项含义

选项	作用
-O2	常规优化级别，平衡运行速度与编译时间
-std=c++17	使用 C++17 标准，与源码中采用的 STL 写法一致
-static	全静态链接，满足 requirement.pdf 中 Compile with all Static link library 及 Executable File 的提交表述
-finput-charset=UTF-8 、 -fexec-charset=UTF-8	源码与可执行文件字符集均为 UTF-8，与源码内中文提示及 Windows 下控制台处理一致

7.2 运行方式与输出

(1) 命令行格式

可执行文件入口为 `main`，支持可选调试输出及两个路径参数，形式如下：

```
PageRank.exe [-v | --verbose] [边表文件路径] [结果输出路径]
```

在 Windows PowerShell 中，当工作目录与 `PageRank.exe` 所在目录相同时，需使用 `./PageRank.exe` 调用。

(2) 默认路径与当前工作目录

未给出路径参数时，程序默认读取当前工作目录下的 `Data.txt`，并写入 `Res.txt`。因此命令行中的相对路径均相对于启动进程时的当前工作目录，与可执行文件所在目录无必然关系；从 源码 或 可执行文件 目录启动时，需相应调整 `Data.txt`、`Res.txt` 的相对或绝对写法。

(3) 常用运行示例

- 在 `Cpp 作业工作区\可执行文件` 目录下，数据位于作业包根目录的 `Data.txt`、结果写入 `Cpp 作业工作区\实验结果\Res.txt` 时，需要采用对应的相对关系：

```
PageRank.exe "../../Data.txt" "../实验结果/Res.txt"
```

- 工作目录为 `pagerank 大作业`、可执行文件位于 `Cpp 作业工作区\源码` 时，例如：

```
"Cpp作业工作区/源码/PageRank.exe" "Data.txt" "Cpp作业工作区/实验结果/Res.txt"
```

(4) 可选详细日志

传入 `-v` 或 `--verbose`，或设置环境变量 `PAGERANK_VERBOSE` 为非空字符串时，程序可向标准错误输出收敛轮次与 L1 残差等信息，在正式评测场景下可不开启。

(5) 输出文件约定

写入的 `Res.txt` 共 100 行，每行 `NodeID`、空格、`Score`；分数为浮点，源码中按 8 位小数输出。`Teleport` 参数 $\alpha = 0.85$ 在源码常量 `kAlpha` 中固定，与 `requirement.pdf` 中「必须报告 0.85」的 `Res.txt` 要求一致。输入边表无法打开或为空时，程序以非零退出码结束。

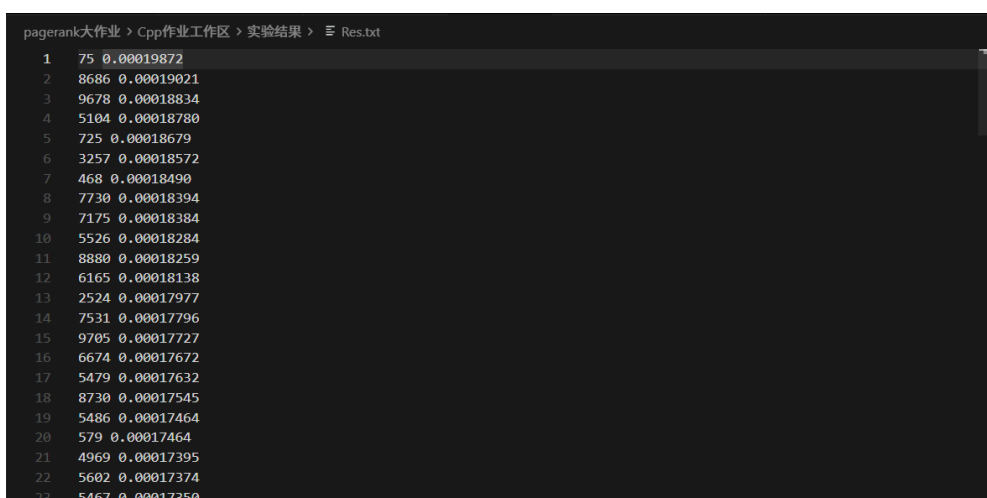
8 实验结果与分析

8.1 Top 100 结果与分析

(1) 提交物与参数

正式提交的 `实验结果\Res.txt` 须在 $\alpha = 0.85$ 下由最终 `PageRank.exe` 生成，共 100 行，格式为 `NodeID Score`（空格分隔）。下表摘自我们所生成的实验结果 `Cpp 作业工作区\实验结果\Res.txt`。

(2) 前十名节点



```

1 75 0.00019872
2 8686 0.00019021
3 9678 0.00018834
4 5104 0.00018780
5 725 0.00018679
6 3257 0.00018572
7 468 0.00018490
8 7730 0.00018394
9 7175 0.00018384
10 5526 0.00018284
11 8880 0.00018259
12 6165 0.00018138
13 2524 0.00017977
14 7531 0.00017796
15 9705 0.00017727
16 6674 0.00017672
17 5479 0.00017632
18 8730 0.00017545
19 5486 0.00017464
20 579 0.00017464
21 4969 0.00017395
22 5602 0.00017374
23 5467 0.00017350

```

图 8.4: Res.txt

排名	NodeID	PageRank 分数
1	75	0.00019872
2	8686	0.00019021
3	9678	0.00018834
4	5104	0.00018780
5	725	0.00018679
6	3257	0.00018572
7	468	0.00018490
8	7730	0.00018394
9	7175	0.00018384
10	5526	0.00018284

(3) 简要分析

- 量级：**顶点数 $n = |V| = 9500$ ，若秩向量接近均匀，则每个分量约为 $1/n \approx 1.05 \times 10^{-4}$ 。表中第 1 名约为 1.99×10^{-4} ，约为上述参考值的 **1.9** 倍，说明前几名的 PageRank 高于均匀情形，但数值仍处在同一数量级；相邻名次之间的分数差距多在约 **1% ~ 5%**，随名次下降变化平缓。

- **编号形态**: NodeID 不连续、跨度大, 与第 3 节统计一致; **同时**按 **NodeID** 升序排序, 表内次序可复现。

8.2 收敛性

(1) 停机准则与源码参数

迭代停止条件为相邻两轮秩向量的 **L1 距离**小于阈值 **kL1Tol** (**1e-10**), 并受 **kMaxIter** 上界保护, 采用 **L1** 残差便于实现与观察。

(2) 记录收敛信息的命令示例 (PowerShell)

在 **Cpp** 作业工作区\可执行文件 目录打开终端, 将标准错误中的收敛信息打印出来, 同时把 **Top 100** 写入单独文件(如 **Res_convergence_run.txt**)以免覆盖提交用 **Res.txt**。在 **Windows PowerShell** 中, 当前目录下的可执行文件须加 **.** 或 **./** 前缀。

```
1 ./PageRank.exe -v "../Data.txt" "../实验结果/Res_convergence_run.txt"
```

(3) 收敛输出摘录

在 (2) 中的命令下得到的输出如下:

```
PS D:\大数据计算及应用\实验一\pagerank大作业\Cpp作业工作区\可执行文件>
./PageRank.exe -v "../Data.txt" "../实验结果/Res_convergence_run.txt"
Converged at iter 15, L1=2.09558e-11
Wrote top 100 to ..\实验结果\Res_convergence_run.txt
PS D:\大数据计算及应用\实验一\pagerank大作业\Cpp作业工作区\可执行文件>
```

图 8.5: 加入-v 参数查看收敛性

```
1 ./PageRank.exe -v "../Data.txt" "../实验结果/Res_convergence_run.txt"
2 Converged at iter 15, L1=2.09558e-11
```

阈值与轮次的工程含义:

本次运行在第 15 轮迭代后满足 $L1 < kL1Tol$ ($kL1Tol = 1e-10$), 停机时 **L1** 约为 2.10×10^{-11} , 已明显小于设定阈值。说明在当前数据上, **1e-10** 作为停机条件偏严格; 若适当放大阈值, 迭代次数会减少, 靠后名次在极端情况下可能略有变动, 需要在精度与计算量之间自行取舍。

8.3 不同 teleport 参数对比

在必须提交的 $\alpha = 0.85$ 结果之外, 我们对其它 **teleport** 作对比。本组在 **PageRank.cpp** 中临时修改 **kAlpha**, 分别编译为 **PageRank_080.exe**、**PageRank_090.exe**, 运行得到 **实验结果\Res_080.txt**、**Res_090.txt**; 随后将 **kAlpha** 恢复为 **0.85**, 重新编译 **PageRank.exe** 并生成提交用 **实验结果\Res.txt**。

(1) 复现命令 (在 **Cpp** 作业工作区\源码 目录、PowerShell 下)

各次编译前须已将源码中 **kAlpha** 改为 **0.80 / 0.90 / 0.85** 再执行对应 **g++** 行。

```
1 g++ -O2 -std=c++17 -static -finput-charset=UTF-8 -fexec-charset=UTF-8 PageRank.cpp -o PageRank_080.exe
2 ./PageRank_080.exe "../Data.txt" "../实验结果/Res_080.txt"
3
4 g++ -O2 -std=c++17 -static -finput-charset=UTF-8 -fexec-charset=UTF-8 PageRank.cpp -o PageRank_090.exe
5 ./PageRank_090.exe "../Data.txt" "../实验结果/Res_090.txt"
6
7 g++ -O2 -std=c++17 -static -finput-charset=UTF-8 -fexec-charset=UTF-8 PageRank.cpp -o PageRank.exe
8 ./PageRank.exe "../Data.txt" "../实验结果/Res.txt"
```

```

PS D:\大数据计算及应用\实验一\pagerank大作业\Cpp作业工作区\源码> g++ -O2 -std=c++17 -static -finput-charset=UTF-8 -fexec-charset=UTF-8 PageRank.cpp -o PageRank_080.exe
PS D:\大数据计算及应用\实验一\pagerank大作业\Cpp作业工作区\源码> ./PageRank_080.exe "..\..\Data\实验结果\Res_080.txt"
PS D:\大数据计算及应用\实验一\pagerank大作业\Cpp作业工作区\源码> g++ -O2 -std=c++17 -static -finput-charset=UTF-8 -fexec-charset=UTF-8 PageRank.cpp -o PageRank_090.exe
PS D:\大数据计算及应用\实验一\pagerank大作业\Cpp作业工作区\源码> ./PageRank_090.exe "..\..\Data.txt" "..\实验结果\Res_090.txt"
PS D:\大数据计算及应用\实验一\pagerank大作业\Cpp作业工作区\源码> g++ -O2 -std=c++17 -static -finput-charset=UTF-8 -fexec-charset=UTF-8 PageRank.cpp -o PageRank.exe
PS D:\大数据计算及应用\实验一\pagerank大作业\Cpp作业工作区\源码> ./PageRank.exe "..\..\Data.txt" "..\实验结果\Res.txt"

```

图 8.6: 不同 teleport 参数下的编译指令

(2) 收敛轮次与停机 L1 (-v 单次运行)

```

PS D:\大数据计算及应用\实验一\pagerank大作业\Cpp作业工作区\源码> ./PageRank_080.exe -v "..\..\Data.txt" "..\实验结果\Res_convergence_run_080.txt"
Converged at iter 14, L1=4.5833e-11
Wrote top 100 to ..\..\Data\实验结果\Res_convergence_run_080.txt
PS D:\大数据计算及应用\实验一\pagerank大作业\Cpp作业工作区\源码> ./PageRank_090.exe -v "..\..\Data.txt" "..\实验结果\Res_convergence_run_090.txt"
Converged at iter 15, L1=4.93923e-11
Wrote top 100 to ..\..\Data\实验结果\Res_convergence_run_090.txt

```

图 8.7: 不同 teleport 参数下的收敛参数

α	收敛迭代轮次	停机时 L1 残差
0.80	14	4.58×10^{-11}
0.85	15	2.10×10^{-11}
0.90	15	4.94×10^{-11}

三者均在 `kL1To1 = 1e-10` 阈值内停止。 $\alpha = 0.80$ 时迭代 14 轮，略少于 0.85 与 0.90 的 15 轮；在其它条件不变时，迭代轮次会随 α 与图结构略有差异。

(3) Top 5 名次与分数 (自各 `Res_*.txt` 摘录)

排名	$\alpha = 0.80$ (NodeID, 分数)	$\alpha = 0.85$	$\alpha = 0.90$
1	75, 0.00019282	75, 0.00019872	75, 0.00020469
2	8686, 0.00018514	8686, 0.00019021	8686, 0.00019527
3	5104, 0.00018307	9678, 0.00018834	9678, 0.00019405
4	9678, 0.00018273	5104, 0.00018780	5104, 0.00019250
5	725, 0.00018233	725, 0.00018679	725, 0.00019121

(4) 对比结论

- **前几名:** 三种 α 下第 1 名均为 75，第 2 名均为 8686，说明在本数据上头部两名对 α 在所取三个取值范围内不发生变化。
- **名次差异:** 第 3、4 名在 $\alpha = 0.80$ 时为 5104、9678；在 $\alpha = 0.85$ 与 0.90 时为 9678、5104，即两顶点交换了先后。 $\alpha = 0.90$ 时第 8、9 名为 7175、7730，与 $\alpha = 0.80$ 、0.85 下的 7730、7175 顺序相反，变化仍局限在相邻名次内（详见各 `Res_*.txt` 前若干行）。
- **分数变化:** Top 5 中出现的同一 NodeID，其 PageRank 分数随 α 从 0.80 增至 0.90 而升高，例如 75 依次为 0.00019282、0.00019872、0.00020469。这与迭代公式中 α 越大、沿原图边

转移的权重越大、随机跳转权重越小的设定一致； α 较小时随机跳转比例更高，同一顶点的分数会略低。

9 总结与心得

本次实验我们在课程给定的数据 `Data.txt` 上，用 C++ 单文件实现了带 `teleport` 与 `dead-end` 处理的 `PageRank`，以 `CSR` 存图，并在迭代中按源点分块遍历，满足作业对稀疏 `Sparse Matrix` 与分块 `Block Matrix` 的要求。结合公式与代码，对幂迭代、`L1` 停机及 α 的含义有了更清楚的认识；并通过对 $\alpha = 0.80 / 0.85 / 0.90$ 的对比，了解了参数变化带来的名次与分数的逐步变化。

并且在我们进行本次实验的过程中，由于目录存在一定的复杂性，因此我非常体会到了相对路径与当前工作目录之间要各自分清楚；在 `PowerShell` 下运行当前目录中的程序需使用 `./PageRank.exe` 等形式；以及使用 `memoryuse-cpp.py` 时要用 `CXXFLAGS` 覆盖脚本默认的 `-fopenmp`，并注意使用 `-static` 选项。让我进一步了解了编译选项、文件的目录关系等底层编译方面的知识与要注意的点，收获很大。

10 参考文献

1. PageRank 原始论文：Sergey Brin, Lawrence Page. *The Anatomy of a Large-Scale Hypertextual Web Search Engine*.
2. 分块/稀疏相关参考链接：<https://zhuanlan.zhihu.com/p/557231877>